

Model Context Protocol is called the USB-C of AI.

It is an open standard developed by folks at Anthropic in Nov 2024 that allows to you to connect AI models / apps to external tools and data.

YOU BUILD A CONNECTOR ONCE AND ANY MCP COMPATIBLE AI APP CAN USE IT.

Anthropic Newsroom

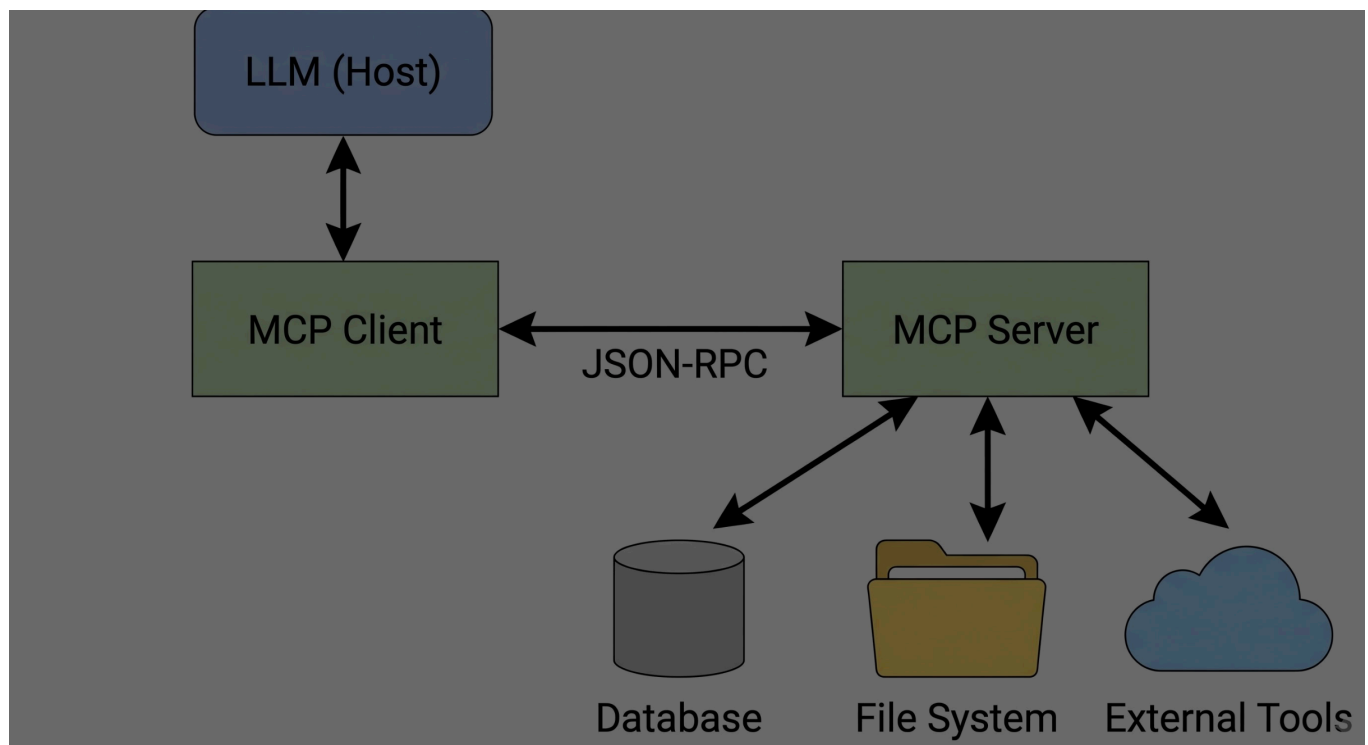
December 2025

anthropic.com

Tuesday

MCP Joins Linux Foundation

In December 2025, Anthropic donated the Model Context Protocol (MCP) to the Linux Foundation. The move places MCP under neutral, community-driven governance, ensuring it remains an open standard for connecting AI models with external tools, applications, and data sources. The initiative is backed by major technology companies including OpenAI, Google, Microsoft, AWS, Cloudflare, Block, and Bloomberg.



How does it work under the hood ?

You have :

1. MCP Host : The AI application itself (Cursor, Claude, VSCode ,etc.)
2. MCP Client : MCP Client lives INSIDE THE HOST. Think of it as the USB C port of your phone (where your phone is the MCP host). It speaks the protocol, manages the connection and you really never need to see it.
3. MCP Server : This is the accessory that you plug in - small program that wraps some tool or data sources or your db , calendar , Github and exposes it in standard MCP format. Don't let the term "server" confuse you, this can be a small 100 line py code acting as server.

-----In simpler terms -----

- **MCP Server** exposes capabilities such as **tools, resources, and prompts**.
- **MCP Host** is the overall AI application, such as an AI assistant, IDE, or agent application.
- **MCP Client** is a component **inside the host** that maintains the connection with a particular MCP server.

For example:

Your AI Application = MCP Host

MCP Client inside your application = communication layer

Google Drive MCP Server = exposes file-search tools

Database MCP Server = exposes SQL tools

When the user asks:

“Find the latest sales report and calculate total revenue.”

The flow is :

User request

↓

MCP Host sends request to the LLM

↓

LLM decides that a tool is required

↓

MCP Client sends the tool request to the MCP Server

↓

MCP Server executes the tool

↓

Result returns through the MCP Client

↓

LLM produces the final answer

What does a server actually give you?

Tools

Resources

Prompts

Tools are what models can do . Resources is what it can read. Prompts are reusable templates.

Why do we need an MCP ?

You're building an AI assistant. You want it to answer questions about your company's data sitting in a Postgres database. The database has an API, but the AI model has no idea how to talk to it. So you write a piece of glue code in the middle:

- it takes what the model wants ("get me overdue loans"),
- turns that into what Postgres actually expects (a SQL query, with the right connection details, the right login),
- and hands the answer back to the model in a form it can read.

That glue code is **an integration**. One AI app, one tool → you write **1** integration. Fine. Manageable.

Now it grows

Your assistant shouldn't only see Postgres. You want it to also read Google Drive, check Slack, and open GitHub issues. Each of those is a separate service with its own login system, its own rules, its own quirks. So you write glue code for each one:

- one for Postgres
- one for Google Drive
- one for Slack
- one for GitHub

One AI app, four tools → 4 integrations. Still okay. Annoying, but okay.

Where it gets ugly ?

Your colleague builds a *different* AI app — say, a customer-support bot. It needs the **same four tools**. But your glue code lives inside *your* app; she can't reuse it. So she writes her own four integrations from scratch.

Now a third team builds an IDE coding assistant. Same four tools. Four more integrations.

Count what just happened:

	Postgres	Drive	Slack	GitHub
Your assistant	glue	glue	glue	glue
Support bot	glue	glue	glue	glue
IDE assistant	glue	glue	glue	glue

That's **3 apps × 4 tools = 12 separate pieces of glue code**.

And every one is maintained by hand. When Slack changes how its login works, three different teams each have to go fix their own Slack glue.

That's the "M×N" — M apps times N tools. The number of integrations is the two multiplied together, and it explodes. 5 apps and 10 tools would be 50.

There is not one MCP server. There is one MCP server *per tool*.

- Postgres has its **own** MCP server.
- Slack has its **own, separate** MCP server.
- GitHub has its **own, separate** MCP server.
- Google Drive has its **own, separate** MCP server.

Four tools → four separate little server programs.

Example

Meet Priya, a portfolio manager at a mutual fund house. She asks the engineering team for an AI assistant she can just talk to — things like *"What's my total exposure to banking stocks across all my equity funds right now?"*

To answer even that one question, the assistant has to reach into three different systems:

1. **The holdings database** — an internal SQL database recording what every fund owns.
2. **The market data service** — an external vendor (think NSE/Bloomberg-style feed) that gives live prices.
3. **The investor records system** — who has invested how much in which fund.

Each of these three systems is a real piece of software with its own **API** — its own way of accepting a request, its own **authentication** (login keys, tokens), and its own **data format** for the reply. None of them was built with an AI model in mind.

Without MCP

The engineering team writes a **custom connector** for each system, by hand, *inside the assistant app*:

- one connector that knows the holdings database's API, its login, its query format;
- one connector that knows the market data vendor's API, its API key, its response format;
- one connector that knows the investor system's API and format.

Three hand-built connectors, living inside this one app. It works — but look at what happens next:

- The market data vendor updates their API. Someone has to open the assistant's code and fix that connector.
- The **compliance team** now wants their *own* AI assistant. It needs the same three systems. But those connectors are buried inside Priya's app — they can't be reused. So the compliance team **rewrites all three connectors from scratch**.

Every new app re-does the same integration work. That's the trap.

With MCP

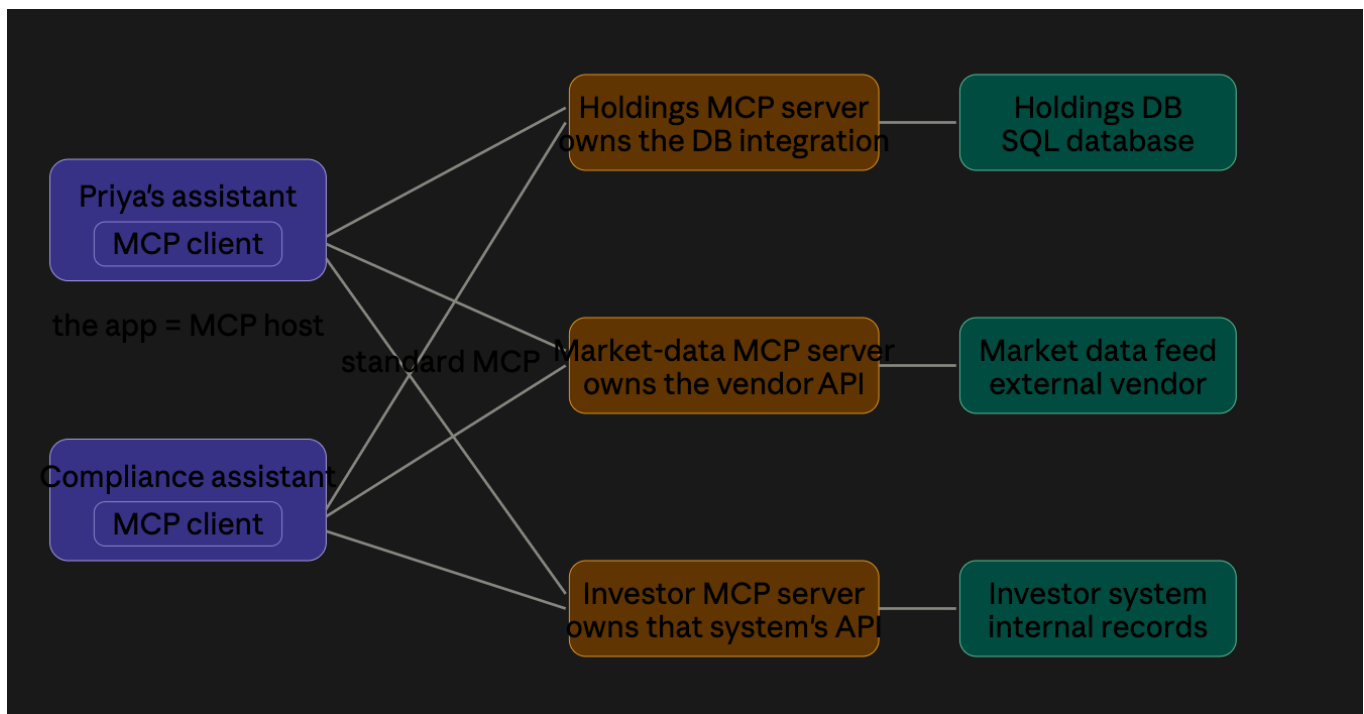
MCP flips the arrangement. Instead of each app building connectors, **each system gets its own MCP server** — a small standalone program that sits in front of exactly one system:

- The **holdings MCP server** knows how to talk to the holdings database (its API, its login). On the outward-facing side, it exposes that capability in **one standard format** defined by MCP.
- The **market data MCP server** does the same for the market data vendor.
- The **investor MCP server** does the same for the investor system.

Each server is written **once**, by whoever owns that system. The messy, system-specific work — the API details, the authentication, the data format — is now sealed inside its own server.

Priya's assistant no longer contains three custom connectors. It contains **one MCP client** — one piece of code that speaks MCP's standard format. That single client connects to all three servers using that one standard. It never needs to know the holdings database's specific API, because the holdings server handles that internally.

And when the compliance team builds their assistant? Their app also just has an MCP client. It connects to the **same three servers** already running. Zero connectors rewritten. That's the win — the integration work was done once, in the servers, and every app reuses it.



MCP server and MCP client

MCP server — a program that sits in front of *one* system and **offers** its capabilities in the standard MCP format. The market-data server offers a `get_price` capability; internally it calls the vendor's API, but the outside world only sees the standard MCP interface. It *waits to be asked*, and it *answers*. One server per system.

MCP client — the component *inside the AI app* that **asks**. It connects to a server, discovers what that server offers, and sends requests in the standard MCP format. Priya's assistant has one; the compliance assistant has its own.

The relationship is the familiar one: **client asks, server answers**. The only twist worth noticing is that one app's client opens a separate connection to *each* server — so one client, many servers.

And the app itself — the thing Priya actually talks to, which holds the model and the MCP client — has a name too: the **MCP host**. Host contains the client; client talks to servers; each server fronts one system. That's the entire architecture.

Without MCP, every AI app hand-builds a connector for every system it needs. With MCP, each system is wrapped once in an MCP **server**, and any app — through its MCP **client** — can use it through one shared standard.

JFK

The fund house switches from **Vendor A to Vendor B** — different API, different login, different data format. This happens constantly in the real world; vendors get dropped all the time.

Without MCP — where do you make the change?

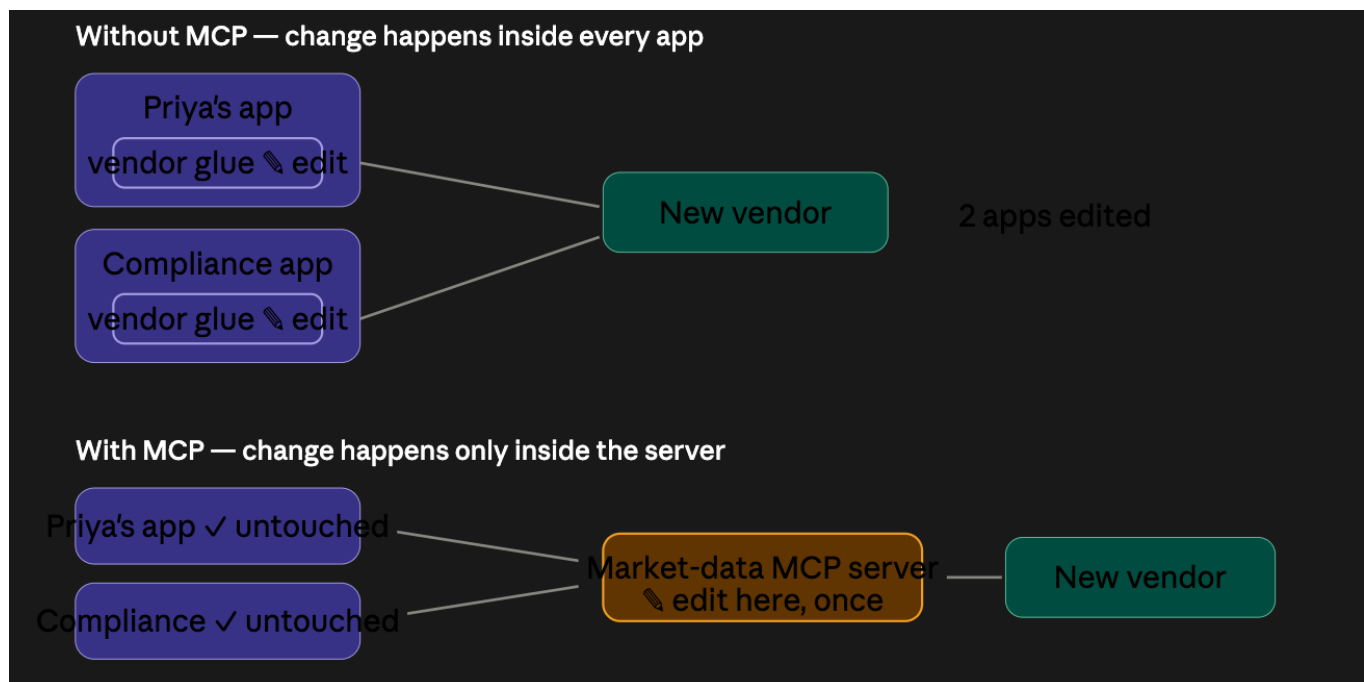
Inside *every app* that talks to prices. You open Priya's app, rip out the Vendor-A glue, write new Vendor-B glue. Then you open the compliance app and do it *again*. Then any third app, again. The change is duplicated across every app, and if one team forgets, that app silently breaks.

The change is scattered.

With MCP — where do you make the change?

In **one place: the market-data MCP server**. You rewrite the inside of that server to call Vendor B instead of Vendor A. Priya's app? Untouched. Compliance app? Untouched. Neither team even needs to be told. Because both apps only ever spoke the standard MCP format to the server — and *that* format didn't change. Only the server's private insides changed.

The changes happen **at the server level, not at the glue level** — because with MCP, there is no glue scattered in the apps anymore. The glue got pulled *out* of every app and consolidated *into* the server, once.



So what is the *use* of an MCP server? Three concrete payoffs

1. **One place to change.** When a system's API changes — new vendor, new login, new data format — you fix the one server. Every app that uses it keeps working, untouched. (The scenario above.)
2. **Write once, reuse everywhere.** The hard integration work is done a single time, inside the server. The second, third, tenth app that needs that system writes *zero* integration code — its client just connects. That's your $M+N$: adding an app doesn't multiply the glue.
3. **The app stops caring how the system works.** Priya's app doesn't know whether prices come from Vendor A, Vendor B, or a spreadsheet. It only knows "there's a market-data

server that answers in the standard format." The messy details are hidden behind the server. That separation is what makes the whole thing maintainable.

MCP versus API

An API is for a programmer who already knows what to call. MCP is for an AI that has to figure out what to call in the moment.

They're not competitors — MCP actually *sits on top of* APIs. Inside almost every MCP server, there's an ordinary API call. MCP wraps it in a standard package a model can understand.

- **The API underneath:** AMFI publishes a data feed at a URL. To use it directly, a *programmer* must know the exact URL, the file format, how to parse semicolon-separated rows. A human hardcodes all that. That's the API way.
- **The MCP server on top:** you wrapped that into a tool called `get_fund_nav` with a plain-English description. Now an AI reads "gets the latest NAV for a fund," decides *on its own* that it needs this, and calls it. Nobody pre-programmed "when the user asks about NAV, hit this URL." The model chose.

When to use which — the simple rule

Use a plain API when a human already decided what happens.

Example: a "Pay Now" button on a checkout page. There's nothing for an AI to figure out — it must charge the card via the payment API. Fixed path, one call, no decision. API.

Use MCP when the AI has to decide what to do based on the conversation.

Example: your Fund Research Desk. The user asks "compare two funds" — the AI has to decide *which* tool to call, with *which* fund, in *what* order. That's a decision, made live. MCP.

The test to teach: **"Did a human already decide what to call, or does the AI have to decide?"**

- Human decided → **API**
- AI decides → **MCP**

MCP doesn't replace APIs — it needs them. Almost every MCP server calls an API inside. If someone says "MCP kills APIs," they've misunderstood. MCP is a *layer*, not a replacement.

Don't use MCP for everything. If you have one fixed call in one app, MCP is pure overhead — just call the API. MCP earns its place when a *model must choose*, and when *many AI apps* reuse the same tools. One app, one fixed call? API every time.

Some common queries around the confusion

1. "If MCP just wraps an API, why not skip it and call the API directly?"

Because with a direct API call, a *human* has to decide in advance exactly what to call and how. That's perfect when the path is fixed — a "Pay Now" button should call the payment API directly, there's nothing to decide. MCP is for when the *model* must decide which capability to use based on what the user said. No decision to make → API. Model has to choose → MCP.

2. "Is MCP going to replace REST APIs?"

No — and anyone who says so has misread the stack. MCP is a *layer above* APIs; almost every MCP server calls a REST API underneath. Your AMFI server proves it: inside it, there's an ordinary call to AMFI's data feed. APIs remain the workhorses; MCP is just a standard way to hand a subset of them to a model.

3. "Isn't MCP the same as function calling / tool calling?"

Related, but not the same — and worth drilling. *Function calling* is the model's ability to emit a structured request to call something. *MCP* is the plumbing that delivers tools to the model in a standard way, including from servers you didn't write. Function calling still happens underneath MCP. Function calling is the model placing an order; MCP is what stocks the menu.

4. "Does my company's data get sent to the AI provider when I use MCP?"

Only what a tool *returns* enters the model's context — so yes, whatever the tool hands back is seen by the model. But the server itself, the database, and the credentials stay on your side; the model never touches them directly. And servers can run entirely inside your own network. This is the question every Edelweiss security person asks first — draw the boundary clearly: the model sees tool *results*, not the systems themselves.

5. "When should I actually build an MCP server instead of just exposing an API?"

Build the server when three things are true: the model needs to *choose* whether to use the capability, more than one AI app will use it, and you expect the toolset to grow. If it's a single fixed call used by a single app, an API is simpler, faster, and more predictable. The honest rule: **MCP is a bet that you'll have many AI apps and many tools. If you won't,**

```
OPENAI_API_KEY="sk-proj-55H4I-  
a08yBF_bw3wxFOzHGIlqIWYUptnWoW55muwwwk8t82Gg2sgwz9sKBybkeY4E4Z8CiUPuTT3  
BibkFJc8QkUoL4orXB_sLd35NaF4MUneJYzFZZhMoKECKi0VHByRM0wLnI-  
DcFISqQ4ymyegXHRZsAwA"
```